Chapter IV – Validation

Instituto Superior de Engenharia
Politécnico de Coimbra

Simão Paredes
sparedes@isec.pt

Outline

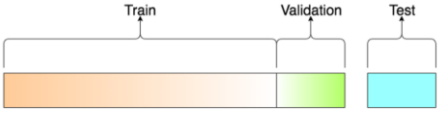
- Validation Approaches
- Regression
- Binary Classification
  - Performance Metrics
    - Confusion matrix
    - Measures
    - ROC Curve
- Multiclass Classification

Validation Approaches

Validation Aproaches

- The validation phase is critical to evaluate the performance of the ML algorithms (classification/regression)
  - To answer the main question

Is this model appropriate for addressing the given problem?



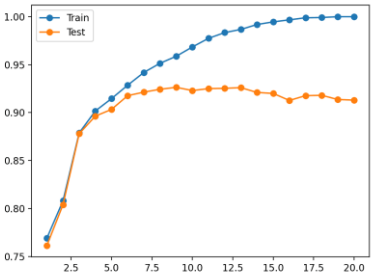
Validation Aproaches

4 different types of validation

- Apparent Validation
- Split-Sample Validation
- Cross - Validation
- Bootstraping Validation

Validation Aproaches

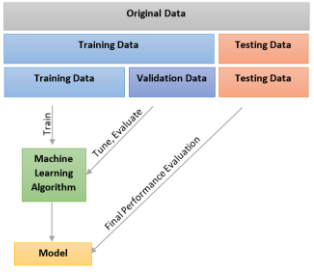
- 4 Types of validation
  - Apparent Validation
    - Training set is the same as the test dataset;
    - This validation may originate a biased performance assessment since the model parameters were optimized for that sample.
      - **Avoid!!**



https://machinelearningmastery.com/overfitting-machine-learning-models/

Validation Aproaches

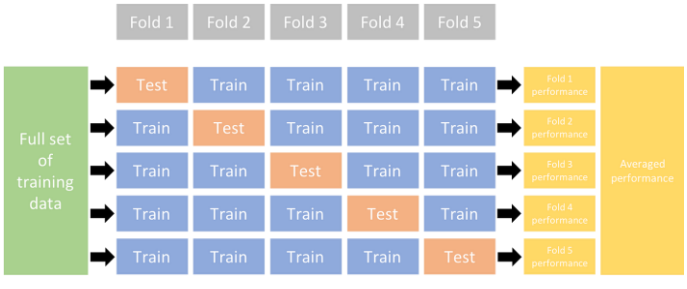
- Split-Sample Validation
  - The dataset is randomly divided in two separate groups (one dedicated to develop the model and the other to validate the model)
    - 70%/30% ; 80%/20%
  - This option has a serious drawback, as the performance assessment is carried out in the entire training set, which may lead to overfitting situations



    - To circumvent this situation a **validation set** should be created.
    - **The training set** is split into a smaller **training set** and a **validation set**.

Validation Aproaches

- Cross -validation
  - the **training set** is divided into distinct subsets (k folds).
  - the model is trained and evaluate in a separate fold, k times, picking a different fold for evaluation every time and training on the other $(k-1)$ folds.

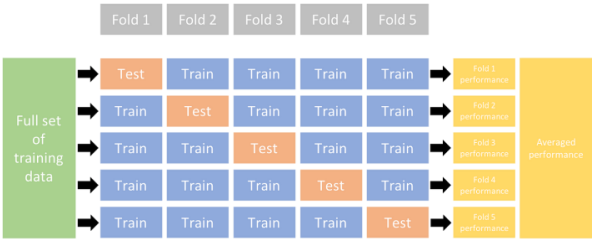


https://docs.ultralytics.com/guides/kfold-cross-validation/

Remember !!

The purpose of cross-validation is **model checking not model building**

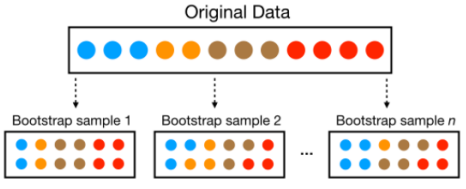
Validation Aproaches



- There are some variants of cross-validation:
  - Apply cross validation **to the training data** is the most correct approach when tuning of hyper parameters is required.
  - However, sometimes due to limitation of the dataset or if the model is trained without adjusting hyper parameters, cross validation is applied to **the whole dataset.**
- After cross-validation, the fit() of the model is done using the entire training dataset

Validation Aproaches

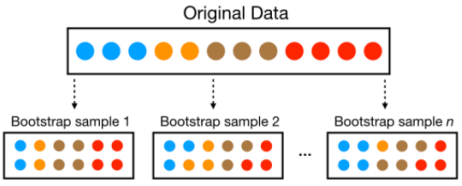
- Bootstrapping Validation
  - This approach is based on the assumption that the **original sample represents the population** from which it was drawn.
    - **Resamples from the original sample** are approximately equivalent to what would be obtained from **multiple samples directly taken from the population.**.



https://bradleyboehmke.github.io/HOML/process.html

Validation Aproaches

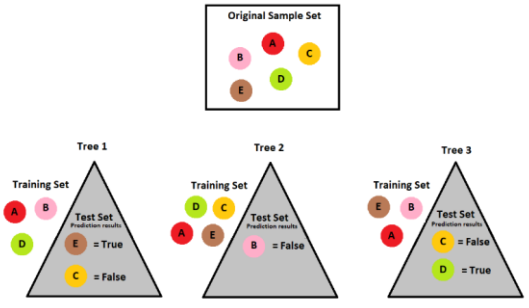
- Bootstrapping Validation
  - The key element are the bootstrap samples that are drawn:
    - from the original sample **with replacement**
      - So, each bootstrap sample is likely to contain duplicate values



    - All the bootstrap samples have the same size:
      - ≤ size_original_sample
      - Resampling technique

Validation Aproaches

- Final statistics are derived through the global analysis of the extracted bootstrap samples.
  - The original observations not contained in a particular bootstrap sample are considered out-of-bag (OOB).



Adopted from:
https://www.analyticsvidhya.com/blog/2020/12/out-of-bag-oob-score-in-the-random-forest-algorithm/

- Ideally, a model is **built on the selected samples** and **validated on the OOB samples**

Validation Aproaches

- Bootstrapping Validation

Definition of the number of bootstrap samples to perform

Usually between 100 and 1000 bootstrap samples

Set a sample size

It depends of the original dataset, but usually with the same size than the original sample

For each bootstrap sample

1. Draw a sample **with replacement,** usually with the same size than the original sample
2. Fit a model on that data sample
3. Estimate the skill of the model on the out-of-bag sample.

"The model is fit on the drawn sample and evaluated on the out-of-bag sample"

Calculate some statistics based on individual models' performances

(each individual model is derived from each sample)

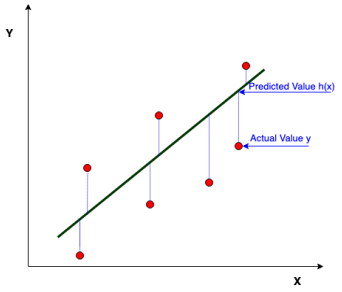
Usually the mean, standard deviation to give a final estimate of the global performance

Regression

Performance Assessment

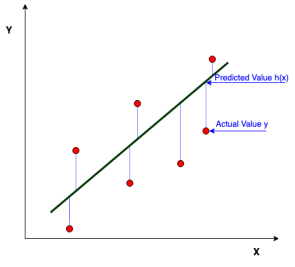
Regression

- The performance assessment is based on the distance between two vectors:
  - $\hat{\mathbf{y}}$ (predictions) | $\mathbf{y}$ (target values)
  - Root Mean Square Error (RMSE) $\text{RMSE}(\mathbf{X}, h) = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(h(\mathbf{x}^{(i)}) - y^{(i)})^2}$
    - $\hat{y}^{(i)} = h(\mathbf{x}^{(i)})$
      - h is the prediction function
      - $\mathbf{X} \in R^{N,M}$
        - N is the number of instances
        - M is the number of features



Regression

- Root Mean Square Error (RMSE) $\text{RMSE}(\mathbf{X}, h) = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(h(\mathbf{x}^{(i)}) - y^{(i)})^2}$



  - It is the standard deviation of the residuals (prediction errors), i.e. how far from the regression line data points are.
    - **The lower the RMSE, the better the prediction.**
      - RMSE is a measure of how concentrated the data is around the line of best fit.

Regression

- Mean Square Error (MSE)

$$\mathrm{MSE}(\mathbf{X}, h) = \frac{1}{N}\sum_{i=1}^{N}(h(\mathbf{x}^{(i)}) - y^{(i)})^2$$

  - It gives more weight to larger differences.
    - **The lower the MSE, the better the prediction.**
  - It has not the same unit as the dependent variable.
  - RMSE and MSE are particularly useful.

Regression

- Mean Absolute Error (MAE)

$$\mathrm{MAE}(\mathbf{X}, h) = \frac{1}{N}\sum_{i=1}^{N}|h(\mathbf{x}^{(i)}) - y^{(i)}|$$

  - It is the average of the difference observed between the predicted and actual values across the whole test set.
    - It considers the absolute value of the difference.
    - A lower value indicates better prediction.

Regression

- R^2 is a different metric to assess the quality of a Linear regression model
  - It is not a metric to assess the error of the model!
  - It is a metric of Goodness-of-Fit
    - R^2 determines the proportion of variance in a dependent variable that can be predicted or explained by independent variables.

$$R^2 = 1 - \frac{\sum_{i=1}^{N}(y^{(i)} - \hat{y}^{(i)})^2}{\sum_{i=1}^{N}(y^{(i)} - \bar{y})^2}$$

$\hat{y}$: predicted value of y
$\bar{y}$: mean value of y

    - R^2 ranges from [0,1]
      - Ideally, the higher the R^2, the more variation is explained by the input variables, and hence better is the model.
        - 0 means that the model explains or predicts 0% of the relationship between the dependent and independent variables
        - A value of 1 indicates that the model predicts 100% of the relationship.

Binary Classification

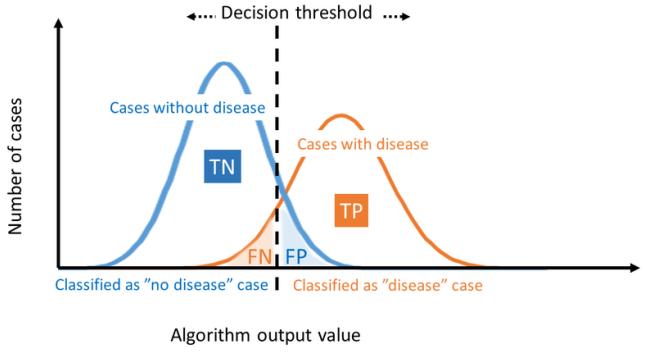
Performance Assessment

Binary Classification

- Binary classification problems
  - based on a set of attributes/feature assign an instance to one of two categories, e.g.:
    - Medical diagnosis for a medical condition (**disease vs. no disease**);
    - Diagnosis (**malignant/benign**) ;
    - Spam filter (**spam/no_spam**);
    - Quality control in industry (**good/not good**);
    - Traffic control (**free/not free**);
    - House Price (**regular/overpriced**),
    - …

Binary Classification

- Binary classification problems
  - The actual output of many binary classification algorithms is a prediction score/value.
    - An instance should be classified as positive or negative, if the value is higher/lower than a classification/decision threshold.

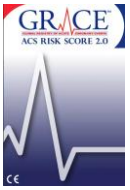


Validation Aproaches

- Actually, the decision threshold of a ML approach is applied similarly to a decision threshold of a statistically derived/based evidence score
  - e.g. GRACE score applied to stratification of Acute Coronary Syndromes patients

TABLE I GRACE RISK SCORE

Variable	Range	Score	Variable	Range	Score
Age (years)	< 40	0	Creatinine (mg/dl)	0 – 0.39	2
	40 – 49	18		0.4 - 0.79	5
	50 – 59	36		0.8 – 1.19	8
	60 – 69	55		1.2 – 1.59	11
	70 – 79	73		1.6 – 1.99	14
	> 80	91		2 – 3.99	23
				>4	31
Heart rate (bpm)	<70	0	Killip class	I	0
	70-89	7		II	21
	90-109	13		III	43
	110-149	23		IV	64
	150-199	36			
	>200	46			
Systolic blood pressure (mmHg)	<80	63	Elevated cardiac markers	No	0
	80-99	58		YES	15
	100-119	47			
	120-139	37			
	140-159	26			
	160-199	11			
	>200	0			
Cardiac arrest at admission	No	0	ST-segment deviation	No	0
	YES	43		YES	30



$high\ risk\ \geq 141$

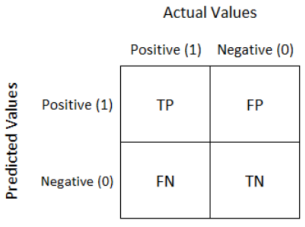
Decision Threshold

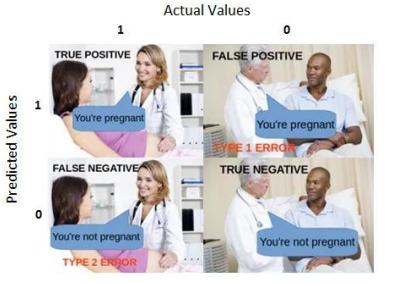
Binary Classification

Performance Assessment
Confusion Matrix & Performance Metrics

Confusion Matrix

- The metrics to properly evaluate a ML model performance are very important
  - Wrong metrics are misleading
- Confusion matrix
  - It is applied for Classification problems where the output can be of two or more types of classes.
    - It is the basis to derive several metrics applied for classification





https://towardsdatascience.com/understanding-confusion-matrix-a9ad42dcfd62

Confusion Matrix

TP
True Positive

True positives are the cases when the **actual class** is positive (1; True) and the **predicted** class is also positive (1; True)

TN
True Negative

True negatives are the cases when the **actual** class is negative (0; False) and the **predicted** class is also negative (0; False)

FP
False Positive

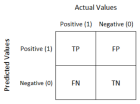
False positives are the cases when the **actual class** is negative and the **predicted** class is positive.

Positive class incorrectly predicted !! (Type I)

FN
False Negative

False negatives are the cases when the **actual** class is positive and the **predicted** class is negative.

Negative class incorrectly predicted !! (Type II)



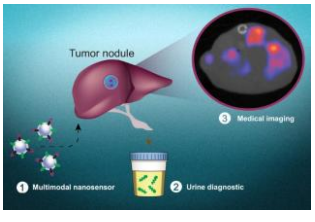
Confusion Matrix

- The ideal scenario is:
  - No False Positives and simultaneously no False Negatives.
    - However with real complex problems that situation is not plausible.
      - Which is more important? False Negatives or False positives?
      - **It depends!**
        - There is not an universal rule that says what should be minimized in all the situations.

		Actual Values: Positive (1)	Actual Values: Negative (0)
Predicted Values	Positive (1)	TP	FP
	Negative (0)	FN	TN

Confusion Matrix

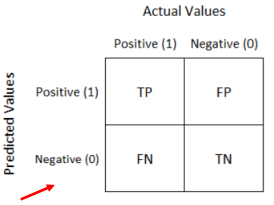
- Minimizing False Negatives:
  - e.g. Cancer detection, it is less dangerous to have a false positive than a false negative
    - Missing a cancer patient will have serious consequences



- Minimizing False Positives:
  - Spam filter, in this situation minimizing false positives is more important than false negatives (avoid important messages in the SPAM folder)
    - If a spam email is not detected it always can be manually deleted.



Confusion Matrix



False Positive (FP)-Also known as a **Type I error**, detects the condition when the condition is absent.

False Negative (FN)-Also known as a **Type II error**, does not detect the condition when the condition is present.

- Which is more important to minimize?



Type II – FN



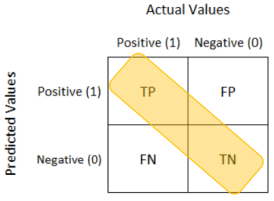
Type I – FP



Type II – FN

Confusion Matrix

- Accuracy



Correct Predictions

- provides the proportion of correct **predictions**

$$A_{cc} = \frac{TP + TN}{TP + FN + FP + TN}$$

- It can be applied when
  - the classes of target variable are approximately balanced.

Confusion Matrix

- When the dataset is (severely) imbalanced

$$A_{cc} = \frac{TP + TN}{TP + FN + FP + TN}$$

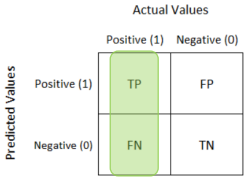
  - Accuracy CANNOT be applied:
    - e.g. cancer detection situation:
      - Only 5 people out of 100 have cancer.
      - Supposing that the model **is terrible and its output is always 0**:
        - Even in that situation the accuracy of such model would be 95%



https://www.analyticsvidhya.com/blog/2020/02/what-is-bootstrap-sampling-in-statistics-and-machine-learning/

Confusion Matrix

- Recall or Sensitivity (SE)



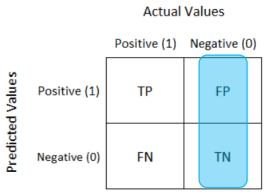
  - provides the proportion of **actual positive** that are successfully **predicted** as positive i.e. the **ability to detect positive** cases

$$SE = \frac{TP}{TP + FN}$$ *Actual Positive*

  - If considered isolated could be misleading.
    - e.g. within 100 people, 5 people actually have cancer. Assuming a bad model that predicts every case as cancer (always 1), so there are no FN, thus the Recall of such model would be 100%.
  - This metric is very useful since combined with other metrics.

Confusion Matrix

- Specificity (SP)



  - provides the proportion of **actual negative** that are successfully **predicted** as negative, i.e. the **ability to detect negative** cases

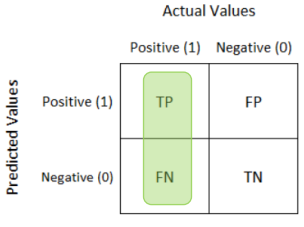
$$SP = \frac{TN}{TN + FP}$$

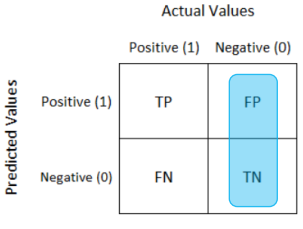
Actual Negative

  - If considered isolated could be misleading.
    - e.g. within 100 people, 5 people actually have cancer. Assuming that the model predicts every case as no cancer (always 0), so there are no FP, the specificity of such model would be 100%.
  - Usually this metric is combined with sensitivity

Confusion Matrix

- Sensitivity and Specificity
  - The simultaneous optimization of sensitivity and specificity is a common strategy to evaluate a binary classifier





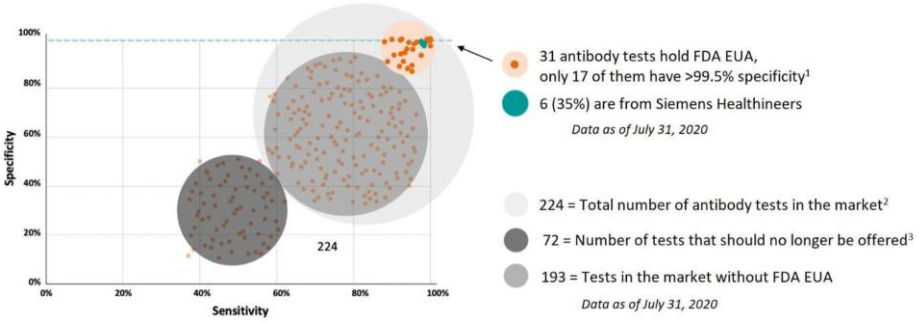
  - Geometric mean provides a single-number evaluation metric which allows to sort different models according their performance (identification of the best model)

$$GM = \sqrt{SE \times SP}$$

Confusion Matrix

- Sensitivity / Specificity
  - The ideal classifier has:

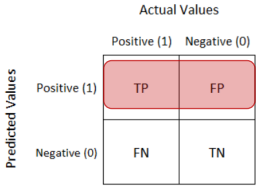
$$SE = \frac{TP}{TP+FN} = 100\% \qquad SP = \frac{TN}{TN+FP} = 100\%$$



https://www.siemens-healthineers.com/pt/laboratory-diagnostics/assays-by-diseases-conditions/infectious-disease-assays/specificity-matters

Confusion Matrix

- Precision



  - provides the proportion of the **predicted** positive cases, that are **actually** positive

$$Precision = \frac{TP}{TP + FP}$$

Predicted Positive

  - If considered isolated could be misleading.
    - e.g: within 100 people, only 5 have cancer. Supposing that model is very bad and predicts only one case as positive and by chance that prediction is correct.
      - So, in this example, the Precision of such model would be 100%.
  - The precision should be considered combined with sensitivity
    - Validate the result obtained through sensitivity

Confusion Matrix

- Precision & Sensitivity (recall)
  - e.g.: within 100 people, only 5 have cancer. Supposing that model is very bad and **predicts all the cases** as Cancer:

$$SE = \frac{TP}{TP + FN} = \frac{5}{5 + 0} = 100\%$$

Unacceptable performance!

$$Precision = \frac{TP}{TP + FP} = \frac{5}{5 + 95} = 5\%$$

  - These two metrics provide different information
    - **Sensitivity**: how many positive were missed?
    - **Precision**: how many positive were caught?

Confusion Matrix

- Precision & Sensitivity (recall)
  - If the focus is on minimizing **False Negatives**:
    - Recall should be as close to 100% as possible **without reducing Precision**

$$SE = \frac{TP}{TP + FN}$$

  - If the focus is on minimizing **False Positives**:
    - Precision should be as close to 100% as possible **without reducing Recall**

$$Precision = \frac{TP}{TP + FP}$$

Trade-off!

Confusion Matrix

- F1 Score
  - Gives information (harmonic mean) about precision and recall:

$$F_1 = 2 \times \frac{precision \times recall}{precision + recall} = \frac{TP}{TP + \frac{1}{2}(FP + FN)}$$

  - Harmonic mean provides better information than regular mean, when:

$recall$ and $precision$ **are very different**

    - In this situation, F_1 tends to approach to the smaller number as compared to the larger value.

Classification metrics



Classification Metrics

- Calculate the following metrics:
  - Accuracy
  - Sensitivity
  - Specificity

Predicted values \ Actual values	Positive (1)	Negative (0)
Positive (1)	1231	146
Negative (0)	394	629

Classification Metrics

- Calculate the following metrics:
  - Precision
  - F1 Score

Predicted values \ Actual values	Positive (1)	Negative (0)
Positive (1)	1231	146
Negative (0)	394	629

Binary Classification

Performance Assessment
Confusion Matrix & Performance Metrics
AUC (Area Under The Curve) ROC (Receiver Operating Characteristics)

Area Under the Curve - ROC

- Area Under the Curve – Receiver Operating Characteristics (AUC - ROC)
  - AUC - ROC curve is a performance measure for classification problem at various thresholds settings and it relates true positive rate (TPR) and false positive rate (FPR)
    - Higher the AUC, better the model

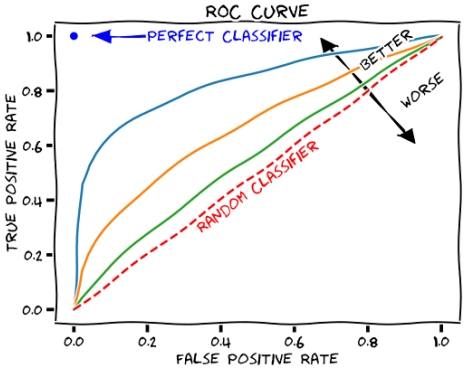
True Positive Rate

$$SE = \frac{TP}{TP + FN}$$

False Positive Rate

$$FPR = 1 - SP$$

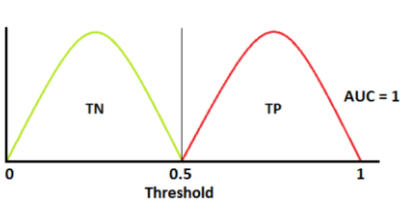
$$FPR = 1 - \frac{TN}{TN + FP}$$

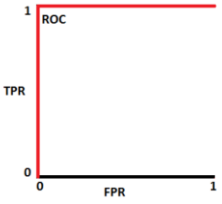


https://commons.wikimedia.org/wiki/File:Roc-draft-xkcd-style.svg

Area Under the Curve - ROC

- An AUC can assume values in interval [0,1]
  - The perfect classifier has an AUC of 1.
    - Ideally, the AUC is 1, the model has an ability to classify positive and negative classes.
    - There is no overlap between the two classes.



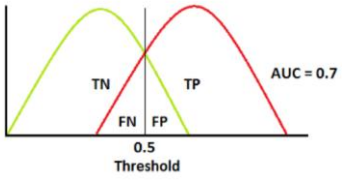


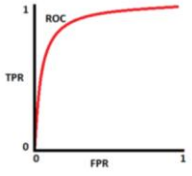
https://towardsdatascience.com/understanding-auc-roc-curve-68b2303cc9c5

  - An AUC of 0.5 means that the model has no class separation capacity
  - An AUC of 0 means that the classifier is reversing the classification outputs
    - 0's should be 1's; 1's should be 0's.

Area Under the Curve - ROC

- Most frequent situation
  - A classification process with some FN and FP.
    - Rarely is observed a perfect separation between the two groups.
      - e.g. An AUC of 0.7, means that there is 70% chance that model will be able to distinguish between positive class and negative class.

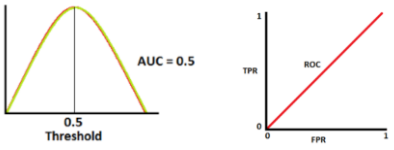




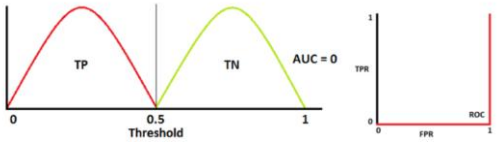
https://towardsdatascience.com/understanding-auc-roc-curve-68b2303cc9c5

Area Under the Curve - ROC

- An AUC of 0.5 means that the model does not have any discrimination capability between positive and negative classes
  - Random Classifier!



- When AUC is approximately 0, the model is actually reversing the classes.
  - Model is predicting negative class as a positive class and vice versa.



Binary Classification

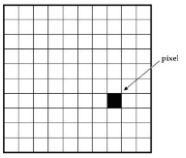
Example (MNIST Dataset)

Binary Classification

- MNIST dataset:
  - 70000 images (digits handwritten) where each image has 784 features (28x28 pixels)

```
1 from sklearn.datasets import fetch_openml
2 mnist=fetch_openml('mnist_784',version=1)
3 mnist
```

https://scikit-learn.org/stable/modules/generated/sklearn.datasets.fetch_openml.html



  - Each **feature** represents the **pixel intensity** from 0 (white) to 255 (black)



Geron A., Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow: Concepts, Tools, and Techniques to Build Intelligent Systems 3rd Edition, O'Reilly 2022

70000 x 784

Binary Classification

- Dataset structure
  - The MNIST stores the attributes values and the respective labels in separate keys:

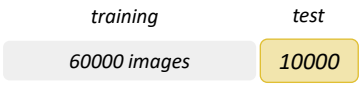
```
{'data': array([[0., 0., 0., ..., 0., 0., 0.],
        [0., 0., 0., ..., 0., 0., 0.],
        [0., 0., 0., ..., 0., 0., 0.],
        ...,
        [0., 0., 0., ..., 0., 0., 0.],
        [0., 0., 0., ..., 0., 0., 0.],
        [0., 0., 0., ..., 0., 0., 0.]]),
 'target': array(['5', '0', '4', ..., '4', '5', '6'], dtype=object),
 'frame': None,
 'feature_names': ['pixel1',
  'pixel2',
  'pixel3',
  'pixel4',
  'pixel5',
```

  - Extraction of of features and labels' values:

```
1 X=mnist['data']
2 y=mnist['target']
```

Binary Classification

- Spliting the whole dataset
  - Training/Test sets:



This MNIST dataset t is already shuffled, so it is possible to create the training and test set this way

```
X_train=X[:60000]
X_test=X[60000:]
y_train=y[:60000]
y_test=y[60000:]
print('training set:', X_train.shape, 'training set label:',y_train.shape)
print('test set:', X_test.shape, 'test set label:',y_test.shape)
```

```
training set: (60000, 784) training set label: (60000,)
test set: (10000, 784) test set label: (10000,)
```


Binary Classification

Selection of classifier (example)

Binary Classification

- 'digit-detector' (identification of a specific digit)
  - Binary problem: two classes in the classification problem
    - e.g. classes: 5 / not-5

Derive a training and a test set based on the proposed binary classification problem:

5/True

Otherwise/False

```
1 y_train_5=(y_train==5)    #true if 5, false otherwise
2 y_test_5=(y_test==5)
```

  - Example:
    - Stochastic Gradient Descent (SGD classifier)
      - default: SVM

Linear classifiers (SVM, logistic regression, etc.) with SGD training.

https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.SGDClassifier.html

Binary Classification

- 'digit-detector' (identification of a specific digit)
  - Training the classifier
    - Model Creation

```
1 from sklearn.linear_model import SGDClassifier
2 sgd_clf=SGDClassifier()
3 sgd_clf.fit(X_train,y_train_5)
```

fit()

```
SGDClassifier(alpha=0.0001, average=False, class_weight=None,
              early_stopping=False, epsilon=0.1, eta0=0.0, fit_intercept=True,
              l1_ratio=0.15, learning_rate='optimal', loss='hinge',
              max_iter=1000, n_iter_no_change=5, n_jobs=None, penalty='l2',
              power_t=0.5, random_state=None, shuffle=True, tol=0.001,
              validation_fraction=0.1, verbose=0, warm_start=False)
```

  - Classifier's Operation

```
1 some_digit=X_train[0]
2 sgd_clf.predict([some_digit])
```

predict()

```
array([ True])
```

The classifier is working but is it good? Is it bad?

Performance Assessment!!

Binary Classification

Selection of classifier (example)
Performance Assessment

Performance Assessment

- Evaluating the performance of a classifier might be more demanding than the evaluation of a regressor
  - There are several possible alternatives that can be applied in some specific situations
    - **Applying cross validation** to evaluate the previous classifier
      - Considering the **training dataset with cross validation**:

```
1 from sklearn.model_selection import cross_val_score
2 cross_val_score(sgf_clf,X_train,y_train_5,cv=3,scoring="accuracy")
```

```
array([0.95035, 0.96035, 0.9604 ])
```

$$\approx 95.7\%$$

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

Performance Assessment

- Is it good??

```
from sklearn.model_selection import cross_val_score
cross_val_score(sgf_clf,X_train,y_train_5,cv=3,scoring="accuracy")
```

```
array([0.95035, 0.96035, 0.9604 ])
```

≈ 95.7%

- Comparing with a really bad classifier!
  - Output is always 0

```
from sklearn.base import BaseEstimator

class AlwaysNot5Classifier(BaseEstimator):
    def fit(self,X,y):
        return self
    def predict(self,X):
        return np.zeros((len(X),1))

never_5=AlwaysNot5Classifier()
cross_val_score(never_5,X_train,y_train_5,cv=3,scoring='accuracy')
```

```
array([0.91125, 0.90855, 0.90915])
```

≈ 90.9%

Creating a classifier whose output is always 0.

Even in that situation, it has an accuracy higher than 90%.

Why?

- Because approximately only 10% of cases are 5's
- In this case (imbalanced datasets) **accuracy is not a reliable metric !!**

Binary Classifier

Performance Assessment

Confusion matrix

Confusion Matrix

- Actual output [y_train_5]
  - y_train_pred is obtained through *cross_val_score()*
    - It returns the prediction made on each test fold (clean prediction as it is made on instances not considered during training)

```
1 from sklearn.metrics import confusion_matrix
2 confusion_matrix(y_train_5,y_train_pred)
```

```
array([[53892,   687],
       [ 1891,  3530]], dtype=int64)
```

Actual \ Predicted	0	1
0	TN 53892	FP 687
1	FN 1891	TP 3530

- Perfect Classifier
  - Unreal situation!

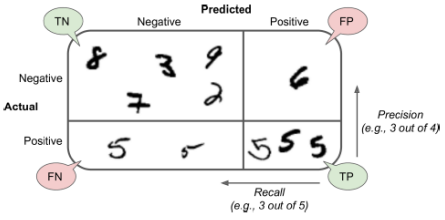
```
1 y_train_pred=y_train_5
2 confusion_matrix(y_train_5,y_train_pred)
```

```
array([[54579,     0],
       [    0,  5421]], dtype=int64)
```

Actual \ Predicted	0	1
0	TN 54579	FP 0
1	FN 0	TP 5421

Confusion Matrix

- Confusion Matrix



Geron A., Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow: Concepts, Tools, and Techniques to Build Intelligent Systems 2rd Edition, O'Reilly 2022

- Alternative to the previous performance Assessment (based on Accuracy)?
  - e.g. **Precision** evaluated along **Recall** (sensitivity)

$$precision = \frac{TP}{TP + FP} \quad ; \quad recall = \frac{TP}{TP + FN}$$

  - Why simultaneous evaluation?
    - Balance between False Positive and False Negative

Binary Classifier

Performance Assessment

Confusion matrix

Precision and Recall

Precision & Recall

- Calculating precision and recall:

```
1 from sklearn.metrics import precision_score, recall_score
2 prc=precision_score(y_train_5,y_train_pred)
3 print('precision :',prc.round(2))
4
5 rcl=recall_score(y_train_5,y_train_pred)
6 print('recall :',rcl.round(2))
```

```
precision : 0.84
recall : 0.65
```

$Accuracy \approx 95.7\%$

- Comparing with the initial value of accuracy
  - It is possible to conclude that the classifier has lower performance than initially suggested by the value of accuracy
  - Actually:
    - It is correct 84% of the positive predictions (5's) (precision)
    - It detects only 65% of the positive instances (in this case 5's) (recall)

Precision & Recall

- An alternative to combine precision and recall in one score is the F1 score

```
1 from sklearn.metrics import f1_score
2 f1s=f1_score(y_train_5,y_train_pred)
3 print('F1 Score :',f1s.round(2))
4
```

```
F1 Score : 0.73
```

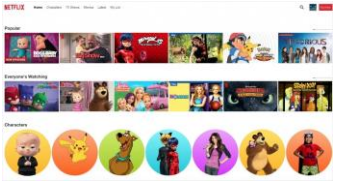
- The classifier only gets a high value of F1 score **if both recall and precision are high**

Precision & Recall

- A trade-off between these two metrics (precision, recall) must be achieved, which one should be favored?
  - It depends!

$$precision = \frac{TP}{TP + FP} \quad ; \quad recall = \frac{TP}{TP + FN}$$

Kids video classification (detect safe?)



High Precision → Reduce FP

Recall ?

Drawback: Reject Safe Videos!

Surveillance images (detect shoplifters?)



High recall → Reduce FN

Precision ?

Drawback : False alerts!

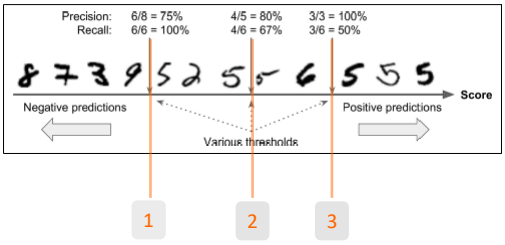
Precision & Recall

- Precision/Recall Trade-off
  - e.g. The SGDClassifier computes a **score** based on a **decision function**

$$IF\ score > threshold \quad THEN\ positive\ class \qquad (5')$$

$$IF\ score \leq threshold \quad THEN\ negative\ class \qquad (no\ 5)$$

  - Assuming the following distribution of instances along the calculated score range



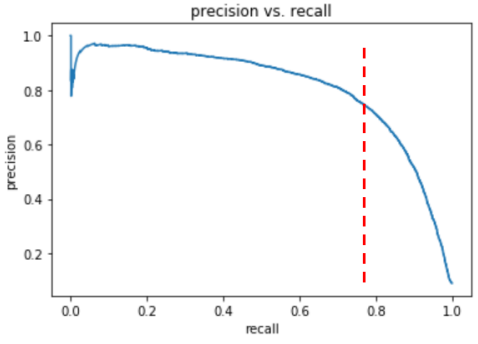
Total of 5's: ***6***

$$precision = \frac{TP}{TP + FP} \uparrow \quad ; \quad recall = \frac{TP}{TP + FN} \downarrow$$

Precision & Recall

- Precision/Recall Trade-off

```
1 plt.plot(recalls, precisions)
2 plt.xlabel('recall')
3 plt.ylabel('precision')
4 plt.title('precision vs. recall');
```



The adjustment of the decision threshold allows the selection of the desired performance.

Important!

A classifier with **very high precision and very low recall** is useless

A classifier with **very high recall and very low precision** is useless

Binary Classifier

Performance Assessment

- Confusion matrix
- Precision and Recall
- ROC Curve

ROC Curve

- ROC Curve
  - As previously mentioned it plots the true positive rate (recall; sensitivity) against the true negative rate (1 – Specificity)
    - The goal is the same as precision/recall, provides a metric in order to allow the creation of a balanced classifier that can achieve a good performance

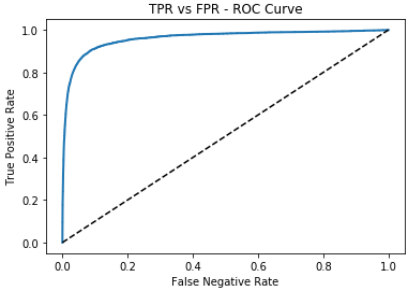
True Positive Rate

$$SE = \frac{TP}{TP + FN}$$

False Positive Rate

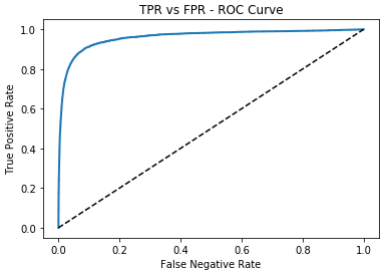
$$FPR = 1 - SP$$

$$FPR = 1 - \frac{TN}{TN + FP}$$



ROC Curve

- ROC Curve
  - The comparison of classifiers can be based on the AUC (area under the curve)
    - Perfect classifier (AUC=1)
    - Purely Random classifier (AUC=0.5)




```
1 from sklearn.metrics import roc_auc_score
2 roc_auc_score(y_train_5,y_scores).round(2)
```

```
0.96
```

  - Based on the ROC curve, it seems that classifier is very good
    - Usually, AUC provides a good assessment of the classifier's performance.

ROC Curve

- Applying a different classifier
  - Random Forest
    - More complex classifier
      - Based on a set of random DT

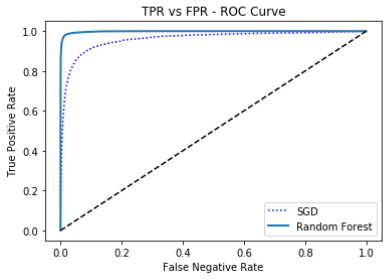
```
1 from sklearn.ensemble import RandomForestClassifier
2 forest_clf=RandomForestClassifier(random_state=42)
3 y_probas_forest=cross_val_predict(forest_clf,X_train,y_train_5,cv=3,method='predict_proba')
```

    - Is it better than the SGD classifier?

ROC Curve

- Classifiers' Performance Comparison
  - Random Forest vs. SGD classifier

```
plt.plot(fpr,tpr,'b:',label='SGD')
plot_roc_curve(fpr_forest,tpr_forest, 'Random Forest')
plt.legend(loc='lower right');
```



Random Forest has a higher AUC

```
roc_auc_score(y_train_5, y_scores_forest)
```

```
0.9983436731328145
```


Multiclass Classification

Multiclass Classification

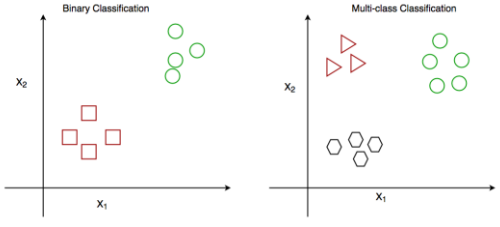
- Multiclass classifiers
  - can distinguish between more than two classes. Each sample is assigned to one and only one class, so it cannot be assigned simultaneously to more than one class
    - Classes are mutually exclusive
      - e.g. Decision Trees; Random Forest, Naïve Bayes are multiclass classifiers.
        - On the contrary, e.g. Support Vector Machine (SVM), Logistic Regression (LR) are binary classifiers.



https://towardsdatascience.com/machine-learning-multiclass-classification-with-imbalanced-data-set-29f6a177c1a

Multiclass Classification

- There are two strategies to **apply Binary Classifiers to multiclass classification** problems
  - One-versus-the-rest (OvR) / One versus One (OvO)
    - OvR



https://www.geeksforgeeks.org/one-vs-rest-strategy-for-multi-class-classification/

    - Three binary classification problems:
      - Classifier 1: **triangles** vs [circles, hexagons]
      - Classifier 2: **circles** vs [triangles, hexagons]
      - Classifier 3: **hexagons** vs [triangles, circles]
    - Number of models:
      - For a multi-class problem with N classes, N models have to be created.

Multiclass Classification

- One-versus-the-rest (OvR):
  - classification of images (0…9) into 10 classes, could be based on the training of **10 binary digit classifier**, one for each digit.



  - Then, the classification of a given image would be based on the scores from **each classifier**, being selected the **classifier with highest score** (one versus all or OvR approach).

Multiclass Classification

- e.g. SVM Classifier

```
somedigit=X[0];

from sklearn.svm import SVC
svm_clf=SVC()
svm_clf.fit(X_train,y_train)
svm_clf.predict([somedigit])
```

```
array([5], dtype=uint8)
```

In this situation the SVM classifier is trained using **y_train** that contains 10 classes (0…9).

Behind the scenes, Scikit learns applies an **OvR strategy**.

- Assessing the respective *decision_function()* for the instance $\mathbf{x}[0]$

```
some_digit_scores=svm_clf.decision_function([some_digit])
```

```
some_digit_scores
```

```
array([[ 1.72501977,  2.72809088,  7.2510018 ,  8.3076379 , -0.31087254,
         9.3132482 ,  1.70975103,  2.76765202,  6.23049537,  4.84771048]])
```

decision_function() returns 10 values, one score per each class

```
np.argmax(some_digit_scores)
```

```
5
```

The highest value of decision function corresponds to class '5'

Multiclass Classification

- Random Forest
  - Performing the same classification but based on a Random Forest

```
somedigit=X[0];
from sklearn.model_selection import cross_val_predict
from sklearn.ensemble import RandomForestClassifier
forest_clf=RandomForestClassifier(random_state=42)
forest_clf.fit(X_train,y_train)
y_probas_forest=cross_val_predict(forest_clf,X_train,y_train,cv=3,method='predict_proba')
```

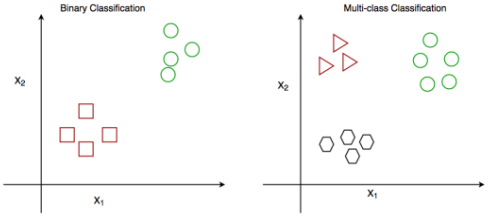
```
y_probas_forest[0,:]
```

```
array([0.  , 0.02, 0.02, 0.26, 0.  , 0.68, 0.  , 0.  , 0.01, 0.01])
```

Random Forest also classifies instance 0 as a '5'

Multiclass Classification

- One versus one (OvO):
  - Classification is based on **the training of a classifier for each pair of categories** (e.g. 1 *vs.* 2; 1 *vs.* 3; ….)



  - Three binary classification problems:
    - Classifier 1: **triangles** vs circles
    - Classifier 2: **triangles** vs hexagons
    - Classifier 3: **circles** vs hexagons
  - Number of Models:
    - If there are N classes then $N \times \frac{(N-1)}{2}$ classifiers are required.

Multiclass Classification

- One versus one (OvO):
  - Each binary classification model predicts one class label and the value (class label) with the **highest number of votes** is predicted by the one-vs-one strategy.



  - The number of classifiers significantly increases with the number of class labels
    - e.g. In the digits example, 45 models will be required to implement this strategy.

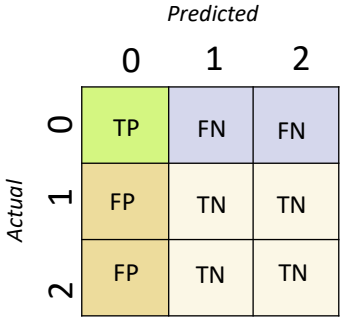
$$N \times \frac{(N-1)}{2}$$

Multiclass Classification

Error Analysis

Error Analysis

- Confusion Matrix for Multiclass Classification
  - Considering a given class, e.g. 0



True Positive: instances **correctly** classified as 0

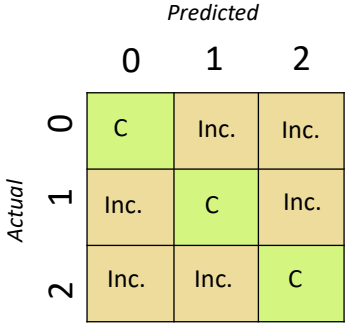
False Positives: instances **incorrectly** classified as 0

False Negative: actual 0's **incorrectly** classified

True Negative: instances **correctly** classified as not 0

Error Analysis

- Confusion Matrix for Multiclass Classification
  - In a global perspective:



  - The confusion matrix is not symetrical

Error Analysis

- Random Forest
  - Confusion Matrix

```
y_train_pred=cross_val_predict(forest_clf,X_train, y_train,cv=3)
```

```
from sklearn.metrics import confusion_matrix
conf_mx=confusion_matrix(y_train,y_train_pred)
```

Predicted

Actual

```
array([[5840,    1,    8,    2,    4,    9,   20,    1,   35,    3],
       [   1, 6635,   43,   11,   13,    5,    6,   13,   12,    3],
       [  27,   12, 5749,   30,   32,    4,   20,   37,   41,    6],
       [   7,    7,   91, 5811,    3,   63,    7,   49,   61,   32],
       [  12,   13,   14,    1, 5645,    0,   28,   14,   17,   98],
       [  20,    9,    9,   64,   12, 5198,   53,    6,   31,   19],
       [  26,   11,    5,    0,   12,   46, 5803,    0,   15,    0],
       [   4,   24,   58,    6,   37,    1,    0, 6037,   11,   87],
       [   9,   35,   44,   52,   26,   52,   27,    5, 5525,   76],
       [  21,   10,   14,   76,   75,   15,    3,   58,   45, 5632]],
      dtype=int64)
```

As expected, the main diagonal contains the highest values, i.e. instances **correctly classified**!

High accuracy
Balanced dataset

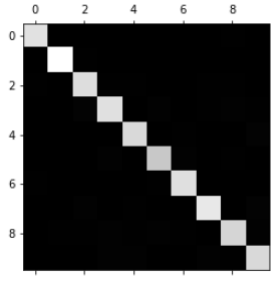
  - The confusion matrix analysis is not so easy as in binary classification
    - A graphical representation may help

Error Analysis

- Random Forest
  - Confusion Matrix
    - Graphical representation (heatmap)

Predicted

Actual




```
import matplotlib.pyplot as plt
plt.matshow(conf_mx, cmap=plt.cm.gray);
```

The **lighter zones** indicate the cells with **higher number of images**.

As expected, according to the performance of Random Forest, the elements of main diagonal indicate that:

- There is a **high number of elements correctly classified**.

Error Analysis

- Confusion Matrix
  - Calculating the number of images in each class (actual values)
    - sum per row

```
row_sums=conf_mx.sum(axis=1,keepdims=True)
row_sums
```

```
array([[5923],
       [6742],
       [5958],
       [6131],
       [5842],
       [5421],
       [5918],
       [6265],
       [5851],
       [5949]], dtype=int64)
```

  - Normalizing the values of confusion matrix, eliminating the influence of the difference in the actual number of images in each category

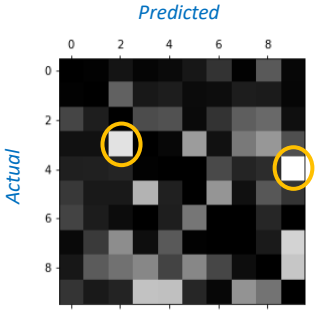
```
norm_conf_mx=conf_mx/row_sums
```


Error Analysis

- Confusion Matrix

Set the elements of main diagonal to 0

```
np.fill_diagonal(norm_conf_mx,0)
plt.matshow(norm_conf_mx,cmap=plt.cm.gray);
```

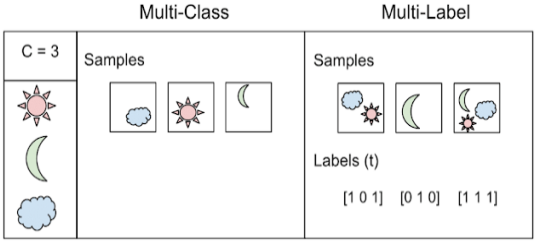


- **Lighter columns** indicate that **many images are incorrectly classified**:
  - In fact, those images are classified as the classes that correspond to those columns:
    - Actual 3 classified as 2
    - Actual 4 classified as 9
- The confusion matrix may often provide some insights about the classification error:
  - Detect some pattern of misclassification / Lighter columns
  - …

Other Types of Classification

Other Types of Classification

- Multi-Label classification
  - This type of classification assigns each instance to more than one label, i.e. predict properties of a sample that are not mutually exclusive
    - Formally, a binary output is assigned to each class (label) for every sample.
      - Positive classes are indicated with 1
      - Negative classes with 0 or -1.



https://prakhartechviz.blogspot.com/2019/02/multi-label-classification-python.html

Other Types of Classification

- Multi-Label classification
  - Example: prediction of the topics relevant to a text document or video.
    - A document or video may be about only **one or some or all** of the following topics:
      - religion
      - politics
      - finance
      - education
      - …



https://unsplash.com/pt-br/fotografias/zootopia-filme-ainda-3A0bs74T8zc

Other Types of Classification

- Multi-output classification
  - each instance is assigned to a set of non-binary properties.
    - The number of targets is higher than 1 and the number of classes per target is greater than 2.
  - This is both a generalization of:
    - multiclass classification task, where there is only one target (label)
    - multilabel classification task, which only considers binary attributes.



Other Types of Classification

- Multi-output classification
  - Example: Classification of fruit images
    - The fruits have two labels and several classes per label
      - Type
        - Apple
        - Pear
        - Orange
      - Color
        - Green
        - Red
        - Yellow
        - Orange.



Other Types of Classification

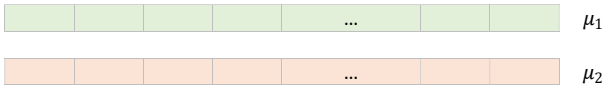
- Summing up:
  - L is the number of labels (target variable)
  - C is the number of classes of the target variable

	C=2	C > 2
L=1	Binary Classification	Multiclass Classification
L > 1	Multilabel Classification	Multioutput Classification

Validation: Statistical Tests

Statistical Tests

- Statistical significance tests are very important
  - They compare the performance of different classifiers operating with the same testing data set.
    - These tests evaluate the evidence against the null hypothesis:
      - H_0: Null hypothesis states that any difference of a given variable extracted from two datasets **is due to chance or sample error**;
      - H_1: Alternative hypothesis, states that there is a reliable difference between the derived variables.



Statistical Tests

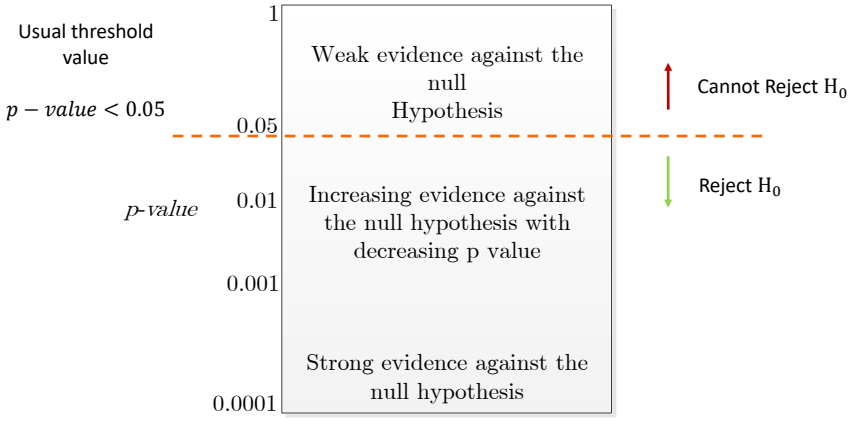
- The significance level of a hypothesis test (*p-value*):
  - It is the **probability of getting** a difference at least as large as the one in the current sample if the classifiers have the same behavior.
    - **Important!**
      - The smaller the *p-value* the stronger the evidence against the null hypothesis

$$p-value \downarrow \quad \Rightarrow \quad strong\ evidence\ against\ H_0 \quad \Rightarrow \quad Reject \quad H_0$$

$$p-value \uparrow \quad \Rightarrow \quad weak\ evidence\ against\ H_0 \quad \Rightarrow \quad Accept \quad H_0$$

Statistical Tests

- Important!
  - The smaller the *p-value* the stronger the evidence against the null hypothesis



Statistical Tests

- Example (non-parametric) Mann-Whitney U test:

```
v=np.arange(0,20,2)
```

```
print(v)
```

```
[ 0  2  4  6  8 10 12 14 16 18]
```

```
u=np.arange(0,300,30)
```

```
print(u)
```

```
[  0  30  60  90 120 150 180 210 240 270]
```

```
stats.mannwhitneyu(u, v)
```

```
MannwhitneyuResult(statistic=90.5, pvalue=0.002487538658093602)
```


Statistical Tests

- The interpretation of the *p-value* may be complemented with the confidence interval (CI) of the **difference** of the variable under analysis:

	model 1	model 2
Performance e.g. Accuracy	acc_{11}	acc_{21}
	acc_{12}	acc_{22}
	...	...
	acc_{1n-1}	acc_{2n-1}
	acc_{1n}	acc_{2n}
	$\boldsymbol{\mu_1}$	$\boldsymbol{\mu_2}$

  - If the 95% confidence interval does not contain the null value:
    - THEN the *p-value* must be smaller than 0.05, therefore **reject H_0**
  - If the 95% confidence interval contains the null value:
    - THEN the *p-value* must be greater than 0.05, therefore **cannot reject H_0**

Statistical Tests

- A hypothesis test never proves that the null hypothesis is true or false
  - It only gives an indication of the strength of the evidence against the null hypothesis:
    - Thus, two types of error can occur:
      - Type I
      - Type II

Significance test	Null hypothesis is true (actual)	Null hypothesis is false (actual)
Reject null hypothesis	Type I error	Correct decision
Do not reject null hypothesis	Correct decision	Type II error

Statistical Tests

Type I error indicates that there is a significant difference between groups when in fact that difference does not exist

False Positive
High Specificity → Low Type I error rate

Significance test	Null hypothesis is true (actual)	Null hypothesis is false (actual)
Reject null hypothesis	Type I error	Correct decision
Do not reject null hypothesis	Correct decision	Type II error

False Negative
High Sensitivity → Low Type II error rate

Type II error indicates a lack of significant difference between groups when in fact that difference exists

Statistical Tests

- Despite these errors (type I/type II) statistical significance tests are very useful:
  - To extract more reliable conclusions on the comparison of the performances of different classifiers (groups of data).
  - There are several statistical tests with different goals.
    - The selection of the test also depends on several aspects such as, the:
      - knowledge about the dataset
        - Data distributions
        - Data type
      - number of dependent variables (label)
      - number of independent variables (features)
      - type of samples

Statistical Tests

- Tests must be selected considering two main criteria:

Paired Samples	Unpaired Samples
Parametric	Non-Parametric

Statistical Tests

- Paired/Unpaired Samples



  - Paired Sample
    - The same sample surveyed at different points in time
      - Pharmaceutical company, wants to analyze if a drug XY helps to lose weight or not.
        - This is done by monitoring the time evolution of 20 people.
  - Unpaired Sample
    - Two different groups/samples
      - Pharmaceutical company, want to analyze if a drug XY helps to lose weight or not.
        - This is done by giving to 20 people the medicine and to 20 **different** people a placebo.

Statistical Tests

Paired Samples

- Some Examples:
  - The before and after effect of a pharmaceutical treatment on the **same group** of people (over time).
  - Body temperature using two different thermometers on **the same group** of participants.
  - Standardized test results of **the same group of students** before and after a study preparation course.
  - Evaluation of two different models **on the same** test set.

Statistical Tests

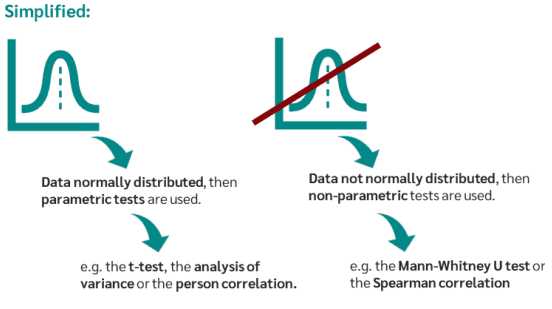
Unpaired Samples

- Some Examples:
  - Research, such as a pharmaceutical study or other treatment plan, where **½ of the subjects are assigned to the treatment group and ½ of the subjects** are randomly assigned to the control group.
  - Research considering **two independent groups**, such as women and men, that examines whether the average bone density is significantly different between the two groups.
  - Comparing the average commuting distance traveled by New York City and San Francisco residents using 1,000 randomly selected participants from each city

https://www.technologynetworks.com/informatics/articles/paired-vs-unpaired-t-test-differences-assumptions-and-hypotheses-330826

Statistical Tests

- Parametric tests
  - The distribution of the population is known and is based on a fixed set of parameters.
- Nonparametric tests
  - The distribution of a population is unknown, and the parameters are not fixed.



https://datatab.net/tutorial/parametric-and-non-parametric-tests

Statistical Tests

- Non-parametric tests are more robust than parametric tests
  - They can be calculated in significantly more situations, e.g.:
    - The true distribution of the random variables is unknown
    - The sample is simply too small to assume a normal distribution.
      - The sample should at least be larger than 30 cases.
- Parametric tests, however, have a greater statistical power than the non-parametric tests.
  - Therefore, if the assumptions for a parametric test are met, **it should always be used**!

Statistical Tests

- Non Parametric tests are more robust but have lower statistical power when compared with parametric tests
  - Example: Mann-Whitney U Test
    - Non parametric, it compares ranks
      - A company wants to know if a new, more expensive training program improves the performance scores of their sales team compared to the old, cheaper program.
      - They randomly assign new employees to one of the two training groups and measure their sales performance score after one month.
        - Group 1: Old Training Program (Scores: 10, 15, 22, 18, 11)
        - Group 2: New Training Program (Scores: 25, 30, 19, 28, 35)

Statistical Tests

- Example: Mann-Whitney U Test
  - Assess if the scores in one group tend to be ranked higher than the scores in the other.
    - All scores from both groups are pooled together and ranked from smallest (rank 1) to the largest (rank n).

Group 1: Old Training Program (Scores: 10, 15, 22, 18, 11)

Group 2: New Training Program (Scores: 25, 30, 19, 28, 35)

Score	Group	Rank
10	Old	1
11	Old	2
15	Old	3
18	Old	4
19	New	5
22	Old	6
25	New	7
28	New	8
30	New	9
35	New	10

Statistical Tests

- Example: Mann-Whitney U Test
  - Ranks for Each Group:
    - Sum of Ranks (Old Training): 1 + 2 + 3 + 4 + 6 = 16
    - Sum of Ranks (New Training): 5 + 7 + 8 + 9 + 10 = 39
  - Calculate the Test Statistic (U)
    - These rank sums are used to calculate a U-statistic for each group.
      - The U statistic is a measure of how much the ranks of one group dominate the ranks of the other group.
    - If the rank sums (and therefore the U-statistic) are significantly different, it suggests that the distributions of the scores are different.
      - The New Training Program likely leads to higher performance scores (since its ranks are consistently higher).

Statistical Tests

- Normality test

Goal	Parametric /Non-Parametric	Test
Normality	Non-parametric	One Sample Kolmogorov-Smirnov test (Kolmogorov-Smirnov goodness-of-fit test)

- Null hypothesis (H_0): Data comes from a Normal Distribution
  - When p-value < 0.05 there is a strong evidence against the Null Hypothesis
    - Data doesn't follow the Normal Distribution

Statistical Tests

- **Unpaired Samples**

Goal	Parametric /Non-Parametric	Test
Compare 2 **unpaired** groups	Non-parametric	Chi-square test of independence
		Fisher's exact test
		Wilcoxon / Mann Whitney U
	Parametric	**Unpaired samples t test**
Compare 3 or more **unpaired** groups	Non-parametric	Chi-square test of independence
		Kruskal-Wallis test
	Parametric	**One-way ANOVA**
		Factorial ANOVA

Statistical Tests

- Unpaired samples t test (Parametric):

```
import numpy as np
from scipy import stats
```

```
N=100
a=np.random.random(N)
```

```
b=np.random.random(N)
```

```
t,p=stats.ttest_ind(a, b)
```

```
print('p-value: ', p)
```

```
p-value:  0.5871480835711309
```


Statistical Tests

- Unpaired samples t test (Parametric):

```
import numpy as np
from scipy import stats
```

```
N=100
a=np.random.random(N)
```

```
b=np.random.random(N)+6
```

```
t,p=stats.ttest_ind(a, b)
```

```
print('p-value: ', p)
```

```
p-value:  1.2188871580304015e-210
```

Statistical Tests

- **Paired Samples**

Compare 2 **paired** groups	**Non-parametric**	**McNemar test**
		Sign test
		Wilcoxon signed-rank test
	Parametric	**Paired samples t test**
Compare 3 or more **paired** groups	Non-parametric	Cochran's Q test
		Friedman two-way analysis of variance
	Parametric	**One-way repeated-measures ANOVA**
		Factorial repeated-measures ANOVA

Statistical Tests

- Comparison with a Specific Value test

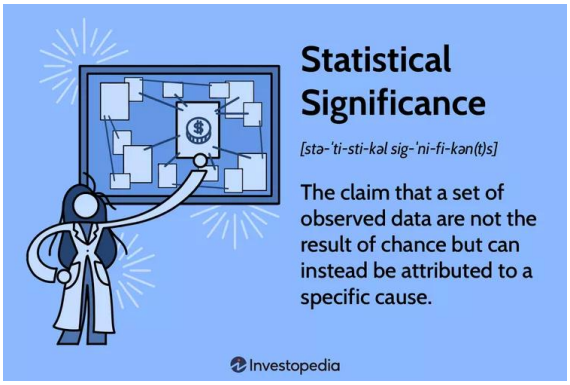
Goal	Parametric /Non-Parametric	Test
Compare one group to a hypothetical value	Non-parametric	Binomial Test
		Chi-square goodness-of-fit test
		Wilcoxon T
	Parametric	One-sample t test

- Association Between Two Variables test

Goal	Parametric /Non-Parametric	Test
Quantify association between Variables	Non-parametric	Kendall correlation
		Spearman correlation
	Parametric	Pearson correlation
		Partial correlation

Statistical Tests

- These tests are commonly applied:
  - To reinforce the conclusions about the classifiers' performance
    - Which one achieves the highest value (considering a specific metric)



https://www.investopedia.com/terms/s/statistical-significance.asp